

AIPL Overview

Actor-based Intelligent Parallel Language — bird’s-eye index

Yasushi Kodama

2026-05-18

Abstract

AIPL は以下の 3 要素を組合せた研究プラットフォーム: (1) マルチランタイム actor 言語 (OCaml + Python + Browser-JS), (2) MAP-Elites 進化計算による次世代言語予測器 (`aice-evolution-v2`), (3) AIPL 自身による型検査・評価系の self-hosting (`aipl-self-host`). 本ドキュメントはプロジェクト全体の俯瞰索引であり, 各アーティファクトとそれらの繋がりを名指しで示す.

Contents

1 プロジェクト構成	1
2 アーキテクチャ図	1
3 Phase 11–17: ランタイム機能	2
4 Phase C–E2: HM 型推論 + 分散ランタイム (2026-05-17 以降)	2
5 Self-host (<code>aipl-self-host/</code>)	3
6 <code>aice-evolution-v2</code> — 次世代言語予測	3
7 型サウンドネスの境界	4
8 クロス言語 remote actors	4
9 推奨読書順	4
10 リポジトリ配置	4
11 現状サマリ	5

1 プロジェクト構成

AIPL は 3 つの柱で構成される:

1. マルチランタイム actor 言語 (OCaml ネイティブ / Python / Browser-JS).
2. 次世代言語予測器 (`aice-evolution-v2`): GA で言語設計の普遍軸を経験的に発見する.
3. Self-host: AIPL のパース・型検査・評価・スケジューラを AIPL 自身で実装したもの (`aipl-self-host/`).

3 ランタイムは HTTP `/api/json/call` で相互運用可能で, `docker/cross/` に Python ↔ OCaml の cross-language remote-actor デモが収録されている.

2 アーキテクチャ図

```
[aice-evolution-v2 (next-gen prediction)]
  .aice spec -----> MAP-Elites GA -----> elite_map / lineage / report

[src/python-aipl (research runtime)]
  lexer (Lark grammar) --> parser --> AST --> aipl_typeck (Phase 11-16)
                                     |
                                     v
                                aipl_interp (actor + scheduler + builtins)

[src/ (OCaml runtime)]
  lexer.mll --> parser.mly --> infer.ml (HM-style) --> eval_thread.ml

[aipl-self-host (AIPL in AIPL)]
  Level A: metacircular eval
    --> B-1 typeck --> B-2 effects --> B-3 channels
    --> B-4 linear --> B-5 owned
    --> C lexer+parser --> C-2 actor scheduler --> C-3 now/Reply

[docker/cross (cross-language remote actors)]
  aipl-python:cross <--> aipl-ocaml:cross via /api/json/call

[aipl_dist (AIPL v2 Distributed runtime, Phase E後)]
  AIPL_DIST_ENABLE=1 で 8 opt-in 機能を重ね合わせ
  既存 .aipl / .abcl は無改変
```

3 Phase 11-17: ランタイム機能

Python ランタイムの主要 Phase と対応サンプル:

phase	feature	source / sample
11a-e	typed annotations / unions / generics / length-arrays	Typecheck*.abcl
12	capability effects <code>!{fs,ai,net,mu}</code>	Effects.abcl
13	CSP channels	Channels.abcl
14	linear / use-after-move	Linear.abcl
15	owned (pub field visibility)	Owned.abcl
16	transient cast at any-boundary	Transient.abcl
17	structured concurrency (<code>scope { future ... }</code>)	Phase17_StructuredConc.abcl

各 Phase に `_test_*.py` 単体テスト (src/python-aipl/) が付属.

4 Phase C-E2: HM 型推論 + 分散ランタイム (2026-05-17 以降)

Phase 11 系の gradual 静的型検査に加え,Python 版 AIPL は Phase C 以降で **constraint-based Hindley-Milner 型推論 + Z3 refinement** を備えるよう進化した. 実装ファイル `src/python-aipl/aipl_inference.py` (約 1255 LOC).

phase	feature	関連レポート
C	基本 HM + Z3 refinement (Int)	PHASE_C_REPORT.md
D-1	Cross-class 推論	PHASE_D_REPORT.md
E- α	where 句を AIPL 表層に追加	PHASE_E_REPORT.md
E- β	Actor field 共有 (method 横断)	PHASE_E_BETA_REPORT.md
E- γ	Record structural typing	PHASE_E_GAMMA_REPORT.md
E- γ -R	Real/Rat refinement (Z3 Real 理論)	PHASE_E_GAMMA_R_REPORT.md
E-2	typeck $\times$ inference 統合 CLI <code>--check</code>	PHASE_E_2_REPORT.md

これと並行して, 進化計算 (Run 1 + Run 2) で発見された分散ランタイム設計を `aipl_dist.py` (591 LOC) として実装:

機能	env var	説明
I-1 routing	<code>AIPL_ROUTE</code>	actor 名 → tag テーブル
I-2 structured_log	<code>AIPL_DIST_LOG_FILE</code>	ND-JSON 1 行 / event
I-3 to-ken_budget	<code>AIPL_DIST_RPM / TPM</code>	60 秒スライディング窓
I-4 checkpoint	<code>AIPL_DIST_CHECKPOINT_DIR</code>	atomic save/restore
IQ quarantine	<code>AIPL_DIST_QUARANTINE_TTL</code>	自動隔離
IM-1 quorum	<code>AIPL_DIST_QUORUM_PROVIDERS</code>	並列 multi-provider
IM-2 subtree	<code>AIPL_DIST_SUBTREE_QUARANTINE</code>	Erlang OTP blast-radius

詳細は `USER_MANUAL.md` §6 / §7 (PDF: `docs/AIPL_User_Manual.pdf`) を参照.

5 Self-host (`aipl-self-host/`)

Level	AIPL 自身は何をするか	smoke
A	<code>eval(ast, env)</code> metacircular	4/4
B-1	Phase 11 型検査器	6/6
B-2	Phase 12 capability-effects 検査器	4/4
B-3	Phase 13 channel element-type 検査器	5/5
B-4	Phase 14 linear / use-after-move	4/4
B-5	Phase 15 owned (pub 可視性)	4/4
C	lexer + parser + eval pipeline	3/3
C-2	actor scheduler + mailbox	6/6
C-3	+ now / Reply 同期返信	1/1
合計		37/37 smoke

各 Level B-N の検査器は `SampleSelfConsistency.abcl` で自身にも適用済—phase monotonicity を実証.

6 aice-evolution-v2 — 次世代言語予測

`aice-evolution-v2/` の MAP-Elites GA は `.aice` 仕様を入力に取り, cross-task 評価 + LLM-directed mutation で言語設計の普遍軸を経験的に導出する.

<code>.aice</code>	予測内容
<code>NextLanguagePrediction.aice</code>	Phase 9 baseline (5 普遍軸)
<code>AIPLSelfHost_A_Metacircular.aice</code> 他	self-host 設計空間 (Level 別)
<code>AIPLPostC2_NextGen.aice</code>	Phase 17 候補 (post-Level-C-2 seed)
<code>AIPLPhase17_Structured.aice</code>	Phase 17 structured-concurrency 精緻化
<code>AIPL_v2_TypeInference.aice</code>	Phase C-E2 (型推論強化)
<code>AIPL_v2_Distributed.aice</code>	I0003/I0023/I0036 (ランタイム層分散化)

Phase 9 予測 (実装済) 5 軸すべて Phase 11–15 として実装:

<code>type_safety</code>	= high / dependent	→ Phase 11
<code>effect_handling</code>	= capability	→ Phase 12
<code>concurrency_model</code>	= csp_channels	→ Phase 13

ownership_model	= linear	-> Phase 14
state_representation	= symbol_owned	-> Phase 15

Phase 17 予測 (実装済) post-Level-C-2 seed で再走した時の trend top 3:

concurrency_model	+0.50	<- 最強 (= structured)
type_safety	+0.25	
ownership_model	+0.25	

elite は concurrency_model = structured + ownership_model = linear + type_safety = dependent で、これがそのまま Phase 17 (scope { future ... } linear future) として実装された。

Phase E-2 後 (Distributed 設計探索) AIPL_v2_Distributed.aice を Run 1 + Run 2 (各 24 min, 1400+ LLM calls) で探索した結果:

- Run 2 throughput_first 勝者: I0036 (Erlang OTP コア, restart_subtree + quorum_replicate + multi_process_local).
- Run 2 hang_resilience 勝者: I0023 (quarantine_and_skip + cluster_with_scheduler).
- Run 2 balanced 勝者: I0003 (低コスト変種, token_budget_aware + same_provider_retry).

3 候補のうち I0003 + IQ + IM-1 + IM-2 が aipl_dist.py として実装済 (上記 Phase C-E2 セクション参照).

可視化 aice-evolution-v2/viz/ 下: index.html (サマリ), level_{A,B1,B2,C}.html (Chart.js ダッシュボード), level_{A,B1,B2,C}_tree.html (SVG 系統樹), postC2.html / phase17.html (post-C2 + 精緻化 run).

7 型サウンドネスの境界

docs/AIPL_Type_Soundness_Report.{tex,pdf} は AIPL の gradual 型システムが何を健全に保証し、どの境界で実行時検査に gracefully 退化するかを論じる. Phase 16 で最優先 unsoundness gap (any → T 境界) を transient cast で塞いだ.

8 クロス言語 remote actors

docker/cross/ は 2 つの Docker image (aipl-python:cross と aipl-ocaml:cross), docker-compose.yml, および fallback の docker run ドライバを収録. 両 image は同一の HTTP wire format (POST /api/json/call) を話すので, Python actor から OCaml actor を呼べる (逆も同様).

9 推奨読書順

新規 reader に推奨する path:

1. USER_MANUAL.md (PDF: docs/AIPL_User_Manual.pdf) — 言語リファレンス §6 (Phase C-E2 型推論) と §7 (AIPL v2 Distributed Runtime / aipl_dist) が post-Phase-11 の新規部分.
2. docs/AIPL_Type_Soundness_Report.pdf — 型システムの証明と未解決部.
3. aipl-self-host/level-a/README.md → level-b{1,2,3,4,5} — AIPL 自身の型システム実装.
4. aipl-self-host/level-c{2,3}/README.md — パーサとスケジューラの self-host.
5. aice-evolution-v2/README.md → docs/AIPL_Phase17_Prediction.md — 次世代軸予測器の中身と現在の推奨.
6. docker/cross/README.md — 言語間相互運用デモ.

10 リポジトリ配置

```
abclcp-project/
  AIPL_OVERVIEW.md          this overview (markdown)
  USER_MANUAL.md           language reference
  README.md                 build instructions
  Makefile                  OCaml + JS build targets
  docs/
    AIPL_Overview.{tex,pdf}  this file's PDF
    AIPL_User_Manual.{tex,pdf} reference PDF
    AIPL_Type_Soundness_Report.{tex,pdf}
    AIPL_Phase17_Prediction.md
    ...
  src/
    *.ml / *.mll / *.mly    OCaml runtime
    python-aipl/            research runtime
      aipl_inference.py      Phase C-E2 HM + Z3
      aipl_dist.py           AIPL v2 Distributed (8 features)
      aipl_typeck.py         Phase 11-16 gradual checker
      aipl_interp.py         interp (+ I-5/I-6 hooks)
      aipl_runtime.py        actor runtime (+ IQ/IM hooks)
    browser-abcl/           browser-side runtime
  abclc/                     OCaml-side .abcl samples
  aice-evolution-v2/         next-gen-axis predictor
    examples/*.aice         GA specs
    src/                    GA implementation
    viz/                    HTML visualisations
  aice-pi-evolution/
    experiments/2026-05-17_aipl_v2_type_inference/
      PHASE_*_REPORT.md      all Phase C-E2 reports
      AIPL_v2_Distributed.{aice,ga,json}
      IMPL_I0003_MVP/        I0003 + IQ + IM impl, tests, samples
  aipl-self-host/           AIPL in AIPL (Levels A -> C-3)
  docker/cross/             cross-language Docker demo
```

11 現状サマリ

2026-05-18 時点で全体スモーク:

- OCaml runtime: 52/52 sample tests (REPL + aipl2c + cc/SDL2)
- Browser JS: 7/7 syntax + 4/4 parse
- Python runtime: 33/33 .abcl + 21/21 .aipl (Phase A-G)
- Distributed cross-language: 8/8 (3-node mock)
- Self-host: 37/37 (Level A → C-3)
- aipl_dist 単体テスト: 27/27
- Distributed MVP デモ: 24/24 (8 feature × 3)

合計 213+ テスト + デモがクリーン. 進化計算 (24 min × 2 run) → I-MVP 実装 (710 LOC)
→ feature 別 24 sample で実演, という一連が完走している.